

Company Booster

Project Budget Tracker & Employee Feedback and Evaluation
Code guide and viva revision notes

Covers	Two features only: Project Budget Tracker and Employee Feedback & Evaluation
Stack	React 19 + Vite, Node.js, Express 5, MongoDB Atlas, Mongoose 9
Database	MongoDB Atlas, database <code>employee_management</code>
Budget code	3 models, 4 services, 1 controller, 1 route file, 11 frontend files
Feedback code	3 models, 5 services, 1 controller, 1 route file, 12 frontend files
External API	Google Gemini — optional, and only ever asked to phrase, never to calculate

How to use this in a viva

Part A is the Budget feature and **Part B** is the Feedback feature. Each part opens with the backend logic written in plain English — read those two sections first, because that is what you will be asked to explain. Then comes the database design, the API list and a table of every function with a one-line description. **Part C** lists every piece of syntax and every method used in these two features with a short explanation of each. **Part D** is a set of likely questions with answers.

Part A

Project Budget Tracker

The feature tracks what a project is spending, predicts what it will finish at, and explains the difference. A team logs time against tasks; each hour is priced using the rate that was in force on the day the work happened; the totals roll up into a forecast with a confidence range.

A1. What makes it different from an ordinary budget screen

- **The forecast leads, not the percentage.** A normal screen shows "55% used". That is a fact about the past. It cannot tell apart a project coasting to a comfortable finish from one that has doubled its burn this fortnight and is already guaranteed to overrun. This screen leads with what the project is *trending to finish at*, and shows the percentage underneath as supporting evidence.
- **The window is rolling, not lifetime.** Everything is computed from the last 14 days of logged time. A lifetime average is dragged towards whatever the team was doing months ago, which is exactly how a project looks calm right up to the moment it does not.
- **The prediction carries a range.** A single predicted number is a confidence nobody has earned. The range is built from how variable the daily burn has actually been, so a steady project gets a narrow band and a lumpy one gets a wide band and deserves it.
- **Rates are effective-dated.** A pay rise adds a new row with its own start date; it never edits the old one. Tools that store a single rate on the employee record quietly rewrite last quarter's margin the moment somebody is promoted.
- **It explains itself without being asked.** Short written notes appear on the page saying what changed and why, rather than sitting inside a report nobody opens.

A2. The backend logic, in plain English

Step 1 — What does one hour cost?

Every time entry has to be turned into money. The rule is a three-level fallback, checked in this order and stopping at the first one that exists:

- 1 **A rate written on the entry itself.** Sometimes a specific piece of work was agreed at a special rate. That always wins.
- 2 **The project's own flat rate.** A fixed-fee engagement may price every hour the same regardless of who works it.
- 3 **The person's own rate, as it stood on the day the work was done.** This is the normal case.

That last point is the important one. The function `rateOn(employee, date)` walks the person's rate history newest-first and returns the first rate whose start date is on or before the day the work happened. So work done three weeks ago is still costed at the old rate even though the person has since had a rise. **Nothing is ever recalculated backwards.**

Say this in the viva

"Cost is resolved per entry, not per person. Entry override beats project rate beats the person's dated rate. And because the person's rate is looked up by the date the work was done, a pay rise never rewrites what past hours cost."

Step 2 — How is the forecast built?

This happens in `buildForecast()`. Five things happen in order:

- 1 **Add up what has been spent.** Every priced entry, summed.
- 2 **Look at only the last 14 days.** Filter the entries to that window.

- 3 **Work out the daily burn.** Every calendar day in the window gets a bucket, *including the days with nothing logged*. A team that worked flat out for three days and rested for eleven is not burning at three days' pace. The average of those 14 buckets is the burn per day.
- 4 **Price the work still outstanding.** The remaining hours come from the task board (unfinished tasks, reduced by how much of each is already done). Those hours are multiplied by the *recent blended rate* — the actual average rate the project has been running at lately.
- 5 **Central estimate = spent + remaining cost.** That is the headline number.

The range is then built from the **coefficient of variation** of the daily burn — the standard deviation divided by the mean. A steady project has a low coefficient and earns a narrow band; an erratic one earns a wide one. The spread is clamped between 8% and 42%, because a project with two data points has not earned a 3% band, and one chaotic week should not produce a range so wide it says nothing.

Confidence is reported as good, fair or thin, and it is about evidence rather than about the answer being comfortable: good needs at least 6 active days in the window and a coefficient under 0.9; fair needs 3 active days; below that it is thin.

Step 3 — How do alerts work?

A project has thresholds, by default 50, 75, 90 and 100 percent. Each is meant to fire **once**, not every day. The budget document therefore stores two lists: `thresholds` (the levels being watched) and `firedThresholds` (the ones already announced). After time is logged, `fireThresholds()` compares the current percentage against the two lists, and anything newly crossed is recorded and then emailed to the people who have been logging time on that project.

- If several levels are crossed at once — one entry can take a project from 45% straight past 50, 75, 90 and 100 — only **one** email is sent, for the highest level. All four are still recorded as fired so none can fire again later.
- The crossing is saved to the database *before* the email is attempted, so a dead mail server costs a notification, never the record of what happened.
- Raising a budget re-arms any threshold the project has dropped back under, so a topped-up project starts warning again on the way up.

Step 4 — The hard stop

A project can be set to refuse new time once it reaches its cap. This is **off by default and deliberately so**: refusing to record work somebody genuinely did makes the ledger a lie, which is usually worse than an overrun. It exists for contracts where the cap is a hard limit, and when it refuses it says exactly why.

Step 5 — The advisor: attributing overspend to a cause

A forecast says where a project is heading. It cannot say *why our budgets keep being wrong*, and that is the question that changes the next one. The advisor looks backwards across every project and produces five kinds of finding:

Finding	What it measures	How
Estimate calibration	How wrong each department's estimates are	For every finished task, spent hours ÷ estimated hours. The median of those ratios per department is the correction factor. Median rather than mean, so one catastrophic task cannot redefine how a whole department estimates.
Worst estimates	Which individual tasks caused the overrun	Tasks that ran more than 25% past their estimate, with the extra hours priced at that task's own blended rate.
Rate efficiency	Expensive people on work that did not need them	Entries at a top-quartile rate on low-priority or sub-four-hour tasks. The saving is measured against the median rate of people who actually worked on the project, not the cheapest possible person, because the cheapest person is rarely a realistic alternative.

Finding	What it measures	How
Shape of spend	Whether the plan held	The project's life is split into four quarters. An even project spends 25% in each. More than 40% in the final quarter is a crunch — which is a different fault from simply being under-budgeted, and needs a different fix.
Billable leakage	Work that was done but never charged	Entries with a cost but no billed amount, as a share of total cost.

Nothing is reported on fewer than **8** finished tasks. That number is printed on the screen rather than buried, because "how much evidence is enough" is exactly the thing a reader should be allowed to disagree with. Below that a pattern is an anecdote.

Step 6 — Where Gemini fits, and where it does not

The advisor can ask Gemini to write the summary paragraph. It is sent **only the findings the code has already computed** — never the raw ledger — so it has nothing to invent a figure from. If there is no API key, or the call fails or times out, a rules-based writer produces the same summary and the page says which one wrote it. **The model phrases; it never calculates.** A model inventing a financial claim is a far worse failure than a plainly worded paragraph.

Step 7 — The what-if simulator

Three decisions, each needing the schedule and the money at the same time:

- **Promise a date.** Works out the margin at that delivery date. Compressing beyond 1.15× the team's sustainable pace is charged a stated 25% overtime premium, taken straight out of margin.
- **Add someone.** Brooks's Law as arithmetic: a new person ramps up 15/50/85/100% across four weeks, and the existing team loses 20% of its time mentoring for the first two. Reports net hours gained, days saved and cost per day saved.
- **Approve leave.** Hours lost per project, how far each finish date slips, which deadlines would be missed, and who else is already off in that window.

A3. The database design

Three collections. The design point is that the **ledger is authoritative**: totals are computed from time entries rather than stored, because a stored total and the work underneath it will disagree the moment anybody touches either one.

ProjectBudget — one per project

Field	Type	What it is for
objective	ObjectId ref	Which project this budget belongs to
totalBudget	Number	The money committed
currency	String	USD by default
thresholds	[Number]	Alert levels being watched, e.g. 50, 75, 90, 100
firedThresholds	[Number]	Levels already announced, so each fires only once
hardStop	Boolean	Refuse new time at the cap. Off by default
projectCostRate / projectBillRate	Number	Optional flat rates that override each person's own
note	String	Why the budget is what it is

Rate — effective-dated, never edited

Field	Type	What it is for
employee	ObjectId ref	Whose rate this is
costRate	Number	What the hour costs the company
billRate	Number	What a client is charged. Zero for internal roles
effectiveFrom	Date	The day this rate starts applying. The key field
reason	String	Why it changed, e.g. "Annual review - promotion"
createdByName	String	Who set it

TimeEntry — the ledger

Field	Type	What it is for
employee / task / objective	ObjectId ref	Who did it, on what, for which project
hours	Number	How long it took
workedOn	Date	The day the work happened. Rates resolve against this , not against today
clockIn / clockOut	Date	An open shift is simply an entry with a clockIn and no clockOut
source	String	'clock' if timed live, 'manual' if logged afterwards
billable	Boolean	Whether a client can be charged for it
costRateOverride / billRateOverride	Number	A rate agreed for this entry specifically
note	String	What was being done

A bug worth being able to describe

task.spentHours originally had **two writers** — the clock-out handler incremented it, and the seed script also wrote it. They had drifted apart on 95 of 120 tasks, and because the performance feature reads that field to score estimate accuracy, the drift was silently lowering people's scores. The fix was to make the ledger the single source of truth: `recomputeSpentHours()` recalculates the total by summing the time entries instead of incrementing. Recomputing is also **idempotent**, so a retried request can no longer inflate the number.

A4. The API

All under `/api/budget`. Static paths are declared before `/project/:id`, so that words like "portfolio" and "rates" can never be read as a project id.

Method and path	What it does
GET <code>/meta</code>	Employees and projects, for the dropdowns
GET <code>/portfolio</code>	Every project with a budget, forecast and totals — the main screen
GET <code>/project/:id</code>	One project in full: forecast, narration, spend by person, burn series, ledger
GET <code>/advisor</code>	The post-mortem and the guidance. Accepts an optional plan to estimate
GET <code>/shift</code>	The open shift for a person, if any
POST <code>/clock-in</code>	Start a shift
POST <code>/clock-out</code>	End a shift and price it
POST <code>/log</code>	Log hours after the fact, dated to when the work happened
GET <code>/entries</code>	The time ledger, filterable
GET <code>/rates</code>	Rate history per person, with the one in force marked
POST <code>/rates</code>	Add a new dated rate. Never edits an old one
POST <code>/budget</code>	Set or update a project's budget and thresholds
GET <code>/simulate/:id/quote</code>	What a promised delivery date is worth
GET <code>/simulate/:id/add-person</code>	What adding a person would actually do
GET <code>/simulate/leave</code>	What a leave request costs the projects

A5. Every function, and what it does

`budget_service.js` — the read model (668 lines)

Pure calculation over whatever was written. It never writes.

Function	What it does
<code>rateOn(employeeId, date, rates)</code>	Finds the rate in force for a person on a given day. Walks history newest-first, returns the first one starting on or before that date
<code>resolveRates(entry, budget, rates)</code>	The three-level fallback: entry override, then project rate, then the person's dated rate. Also reports which one was used
<code>priceEntry(entry, budget, rates)</code>	Turns one time entry into cost and billed amount
<code>buildForecast({...})</code>	The core calculation: spend, 14-day burn, remaining cost, central estimate, confidence range, days to finish
<code>narrate({...})</code>	Writes the short plain-English notes that appear on the page — what changed, and why
<code>evaluateAlerts(budget, percent)</code>	Which thresholds are crossed, and which of those are newly crossed
<code>loadRates()</code>	Loads the whole rate table once, grouped per person, sorted newest-first
<code>loadContext()</code>	Loads employees and rates once for a whole page, so the portfolio does not re-fetch per project
<code>projectFinancials(id, opts)</code>	Everything about one project: forecast, spend by person, daily series, ledger, alerts, narration
<code>portfolio()</code>	Every budgeted project plus company totals. Runs the projects in parallel

Function	What it does
<code>deadlineImpact(task, newDate)</code>	What moving a deadline costs, in money — the join between the task board and the budget
<code>fmt(value, budget)</code>	Formats money with the right currency symbol

time_service.js — the write side (379 lines)

Function	What it does
<code>clockIn({...})</code>	Starts a shift. Refuses if one is already open, or if the project has hit a hard stop
<code>clockOut({...})</code>	Ends the shift, converts elapsed time to hours, recomputes the task total, checks thresholds
<code>logManual({...})</code>	Records hours worked earlier. Dated to when the work happened, so it is costed at that day's rate
<code>openShift(employeeId)</code>	Finds an entry with a clockIn and no clockOut — that is what an open shift is
<code>recomputeSpentHours(taskIds)</code>	Recalculates a task's total hours from the ledger with an aggregate. Idempotent
<code>reconcileAll()</code>	Repair pass for data written before the ledger became authoritative
<code>checkHardStop(objectiveId)</code>	Returns a refusal message if the project is capped and at 100%
<code>fireThresholds(objectiveId)</code>	Records newly crossed levels and emails the people logging time. One email for the highest level
<code>setRate({...})</code>	Adds a new dated rate row. Never edits an existing one
<code>listRates()</code>	Rate history grouped per person, with the rate in force today marked and future ones flagged as scheduled
<code>setBudget({...})</code>	Creates or updates a project budget. Raising it re-arms thresholds the project has dropped under

budget_advisor.js — the analysis (706 lines)

Function	What it does
<code>calibration(doneTasks)</code>	Median and 90th-percentile ratio of spent to estimated hours, per department, plus example tasks
<code>worstEstimates(tasks, cost)</code>	The tasks that ran furthest past their estimate, with the overrun priced
<code>rateEfficiency(entries, tasks)</code>	Senior people on small or low-priority work, and what routing it to the median rate would have saved
<code>shapeOfSpend(entries)</code>	Share of spend in each quarter of the project's life. Over 40% in the last quarter is a crunch
<code>billableLeak(entries)</code>	Work with a cost but nothing billed, as a share of total cost
<code>reviewProject(id, ctx)</code>	Runs all five checks over one project
<code>buildFindings(...)</code>	Turns the raw checks into ranked, severity-tagged findings with a headline figure
<code>budgetFor(plan, rows, rates)</code>	Applies the correction factors to a proposed plan and recommends what to commit
<code>askModel(evidence)</code>	Asks Gemini to phrase the summary, sending only computed findings. Falls back on failure
<code>summariseByRules(...)</code>	Writes the same summary without a model
<code>advise({plan})</code>	The public entry point that assembles all of the above

simulator_service.js — the what-ifs (403 lines)

Function	What it does
<code>quoteDate(id, date)</code>	Margin, cost and feasibility of promising a delivery date
<code>addPerson(id, person, days)</code>	Brooks's Law as arithmetic: ramp-up, mentoring cost, net hours, days saved

Function	What it does
<code>leaveImpact(person, from, to)</code>	Hours lost per project, finish-date slip, deadlines at risk, who else is off
<code>workingDaysBetween(a, b)</code>	Counts weekdays only — weekends are excluded from every day count
<code>addWorkingDays(date, n)</code>	Moves a date forward by n working days

The frontend (11 files)

File	What it renders
<code>budget_layout.jsx</code>	The shell: tabs, the "working as" person selector, and the <code>.bud</code> style scope
<code>budget_context.js</code>	The shared context object and its hook, kept apart from the component that provides them
<code>budget.jsx</code>	The portfolio. Each headline figure opens into the projects it is a sum of
<code>budget_project.jsx</code>	One project: forecast, what changed, burn chart, spend by person, the ledger. Rows expand
<code>budget_clock.jsx</code>	Clock on and off, or log hours after the fact
<code>budget_rates.jsx</code>	Rate history per person, with the one in force marked
<code>budget_simulate.jsx</code>	The three what-if panels
<code>budget_advisor.jsx</code>	Post-mortem, guidance, the calibration chart and the estimator
<code>budget_ui.jsx</code>	Shared components: Icon, Avatar, ForecastBar, BurnChart, Note
<code>budget_format.js</code>	<code>money()</code> , <code>formatDate()</code> , <code>shortDate()</code>
<code>budget.css</code>	Every rule scoped under <code>.bud</code> so it cannot leak into other pages

Part B

Employee Feedback & Evaluation

The feature collects manager, peer, self and client feedback, checks the reviewers as well as the reviewed, proposes development objectives from recurring themes, and keeps an append-only record of everything an automated agent touched.

B1. The four design decisions worth defending

- **One collection for all four sources.** Manager, peer, self and client reviews are the same shape in one collection with a source field, rather than four tables or four tabs. That is what makes them comparable on one set of axes — and the disagreement between them is the finding, not a nuisance.
- **The self-assessment gap is the point.** Where somebody's own polygon sits outside everyone else's, that is a blind spot. Where a client rates delivery well above the internal view, the team is being harder on itself than the people paying for the work are.
- **The agent proposes; a human decides.** Nothing the agent drafts is written into the task feature until a named person approves it, and both the proposal and the approval go into an append-only trust log. The gate is built as a gate, not promised in a comment.
- **Calibration checks the reviewers.** Most systems only measure the people being reviewed. This one also asks whether a reviewer is systematically generous, systematically harsh, gives everyone the same score, or writes in vague language.

B2. The backend logic, in plain English

Step 1 — What a review is

A review has a reviewer, a subject, a source (manager, peer, self or client), a cycle, up to six competency ratings out of 5, and free-text strengths and improvements. The six competencies are Delivery, Quality, Communication, Collaboration, Ownership and Initiative. The overall score is the mean of whichever competencies were actually rated — not of all six — so leaving one blank does not silently count as a zero.

Step 2 — The feedback graph

`buildGraph()` turns one person's reviews into three views:

- 1 **A radar**, with one polygon per source over the six competencies. Four opinions on one set of axes.
- 2 **A timeline**, so a rating can be read as a trend rather than a snapshot.
- 3 **A self-gap**: the difference between what the person said about themselves and what everybody else said, per competency. A positive gap is over-confidence, a negative one is under-selling.

Step 3 — Calibration: checking the reviewers

Four checks, each producing an explainable finding:

Check	What it looks for
Drift	A reviewer whose scores sit consistently above or below what other reviewers gave the same people
Clustering	A reviewer who gives everyone almost the same score, so the ratings carry no information (measured by standard deviation)
Vague language	Comments that describe the person rather than the work, with no example, date or number to anchor them
Distribution	The overall shape of scores across the company, so a lenient culture is visible as a culture rather than blamed on individuals

Two calibration bugs worth being able to describe

Both produced confident-but-wrong output. **First**, drift was originally measured against the *company mean* — which flags anybody who happens to review a weak team as harsh. It is now measured against what *other reviewers gave the same people*, which is the only fair comparison. **Second**, a reviewer's own earlier reviews had to be excluded from that baseline, or they are partly being compared against themselves, which drags every drift figure towards zero and hides real bias.

Step 4 — The agent

`scan()` reads every submitted review and looks for a theme raised independently by **three or more separate reviewers** about the same person. It proposes only the **strongest theme per person** — one objective at a time, because a development plan with six items is not a plan. Each proposal is drafted as an objective with a title, a rationale and the evidence it was built from, and is stored as a `FeedbackSignal` with status `proposed`.

A named human then approves or dismisses it. Only on approval is anything written into the task feature. Every step — proposed, edited, approved, dismissed — is appended to the audit log with who did it and when. A **fingerprint** of the theme prevents the same proposal being raised twice.

Step 5 — Record against reviewers

This is the cross-feature comparison, and the part with the most interesting reasoning behind it. The system holds two completely independent opinions of every person: a **score derived from finished work** that nobody typed in, and a **rating written by colleagues** that nobody computed.

The obvious way to compare them — multiply the rating out of 5 by 20 and put it beside the score out of 100 — is **wrong**, and knowing why is the point. Ratings cluster: almost nobody is given a 1 or a 5, so an average of 3.6 is not "72% of anything", it is the middle of the range people actually use. Rescaling would manufacture disagreement out of nothing but the shape of the two distributions.

So both readings are converted to a **percentile within this company**: where does this person sit relative to their own colleagues on each instrument. That is unit-free, it survives a lenient or a harsh review culture, and it turns the comparison into one of ranks rather than of numbers that were never on the same scale. A gap of more than **20 percentile points** is treated as a disagreement worth reading.

The overall agreement figure is **Spearman's rank correlation**, chosen over Pearson deliberately: nothing here claims the two scales are linearly related, only that two instruments measuring anything alike should put people in a similar order. Ties share the average of the positions they occupy, otherwise two identical scores would be ranked differently by list order alone.

Each disagreement then names **what the recorded data can and cannot see** — for example, that collaboration is only 20% of the derived score while completed tasks are 40%, so somebody who spends their time unblocking others reads as a gap in their own output. Every line is a check that either holds or does not, so a manager can agree or disagree with each one. Where nothing in the data explains the gap, it says so rather than inventing a plausible sentence.

B3. The database design

Review — all four sources in one collection

Field	Type	What it is for
employee	ObjectId ref	Who the feedback is about
reviewer	ObjectId ref	Who wrote it. Null for client feedback
reviewerName / clientName	String	Kept as text so external client feedback needs no user account
source	String enum	manager peer self client — the field that makes one collection work
cycle	String	The review period, e.g. "2026-Q3"
ratings	[[competency, score, note]]	Up to six competencies, each 1–5, each with optional evidence
overall	Number	Mean of the competencies actually rated
strengths / improvements	String	The free text
status	String enum	draft submitted acknowledged
acknowledgedAt	Date	When the subject read it — drives the "not yet read" count

FeedbackSignal — an agent proposal

Field	Type	What it is for
employee	ObjectId ref	Who the theme is about
theme / competency	String	What the reviewers kept raising
evidence	[Object]	The reviews it was drawn from, so the proposal can be checked
draft	Object	The objective the agent would create
status	String enum	proposed approved dismissed
approvedBy / approvedByName	ref / String	The named human who decided
fingerprint	String	Stops the same theme being proposed twice
engine	String	Whether the wording came from Gemini or from the rules

AuditEvent — append-only

Field	Type	What it is for
actorType	String	'agent' or 'human' — so the two can be separated in the log
actorName	String	Who did it
action	String	What happened: proposed, approved, dismissed, created
subject / subjectId	String / ref	What it happened to
detail	Object	Enough context to reconstruct the decision
createdAt	Date	When. Records are never updated or deleted

B4. The API

All under `/api/feedback`.

Method and path	What it does
GET <code>/meta</code>	Employees, competencies and the current cycle, for the forms

Method and path	What it does
GET /overview	Company summary, per-source averages, roster, people awaiting feedback, open proposals
GET /employee/:id	One person's full dossier: radar, timeline, self-gap, every review
GET /reviews	List reviews, filterable by person, source, cycle or status
POST /reviews	Write a review
PATCH /reviews/:id	Edit one
POST /reviews/:id/acknowledge	The subject marks it read
DELETE /reviews/:id	Remove one
GET /calibration	The reviewer findings: drift, clustering, vague language, distribution
GET /reconciliation	The record against the reviewers — percentiles, Spearman, and each gap explained
POST /agent/scan	Run the agent over every review and produce proposals
GET /signals	The agent's proposals
POST /signals/:id/approve	A named human approves; only now is an objective created
POST /signals/:id/dismiss	A named human rejects it, with a reason
GET /audit	The append-only trust log

B5. Every function, and what it does

review_service.js — reviews and the graph (308 lines)

Function	What it does
listReviews(filters)	Reviews matching a person, source, project, cycle or status
getReview(id)	One review in full
buildGraph(reviews)	The radar, the timeline and the self-gap for one person
employeeDossier(id)	Everything about one person: their graph plus every review on record
createReview(body, actor)	Writes a review. Computes the overall from whichever competencies were rated
updateReview(id, body)	Edits one, recomputing the overall
acknowledgeReview(id)	Marks it read by the subject and stamps the time
deleteReview(id)	Removes one
pendingAfterDelivery()	People who finished work on a project and have had nothing said about it
defaultCycle()	The current review period, derived from the date
shape(review)	Turns a database document into the exact shape the frontend expects

calibration_service.js — checking the reviewers (293 lines)

Function	What it does
driftAgainstPeers(...)	Compares a reviewer to what other reviewers gave the same people, excluding their own earlier reviews
analyseReviewers(reviews)	Per reviewer: drift, spread, and how specific their language is
analyseDepartments(reviews)	Whether one department rates differently from the rest
analyseCompetencies(reviews)	Which competencies are rated highest and lowest company-wide
assessLanguage(text)	Flags comments that describe the person rather than the work
buildDistribution(reviews)	The shape of scores across the company
calibration({cycle})	Assembles all the findings for the page

feedback_agent.js — propose, never decide (461 lines)

Function	What it does
scan({employee})	Reads every submitted review, finds themes raised by 3+ reviewers, drafts one objective per person
concernsIn(reviews)	Groups the recurring concerns by competency
draftByRules(theme)	Writes the objective without a model
draftByModel(theme)	Asks Gemini to phrase the same objective. Falls back to the rules on any failure
fingerprintOf(theme)	A stable key so the same proposal cannot be raised twice
listSignals(filters)	The proposals, by status
approveSignal(id, actor)	The gate. Creates the objective, records who approved it, writes to the audit log
dismissSignal(id, actor)	Rejects it with a reason, also audited

reconciliation_service.js — record against reviewers (395 lines)

Function	What it does
<code>percentileOf(value, population)</code>	Where one value sits within the company, 0–100. Half-credits ties
<code>spearman(pairs)</code>	Rank correlation between the two orderings. Ties share the average rank
<code>explain({...})</code>	Names what the recorded data can and cannot see for one disagreement. Says so when nothing explains it
<code>reconcile(options)</code>	The public entry point: reads the performance overview and the reviews once each, converts both to percentiles, ranks the gaps

audit_service.js — the trust log (46 lines)

Function	What it does
<code>record(event)</code>	Appends one event. Nothing in this file updates or deletes
<code>byAgent()</code> / <code>byHuman()</code>	The log split by who acted
<code>list(filters)</code>	The whole trail, newest first

The frontend (12 files)

File	What it renders
<code>feedback_layout.jsx</code>	The shell: tabs, the "acting as" selector, and the <code>.fb</code> style scope
<code>feedback_context.js</code>	The actor context and its hook
<code>feedback.jsx</code>	Company overview: averages per source, who is owed feedback, open proposals, the roster
<code>feedback_profile.jsx</code>	One person: the radar, the timeline, the self-gap, every review
<code>feedback_write.jsx</code>	The review form, with live language checking as you type
<code>feedback_calibration.jsx</code>	The reviewer findings, each showing the comparison that produced it
<code>feedback_reconciliation.jsx</code>	The slope graph, the gap cards and the full table
<code>feedback_agent.jsx</code>	The proposals, and the approve / dismiss gate
<code>feedback_trust.jsx</code>	The append-only audit trail
<code>feedback_ui.jsx</code>	Shared components: Icon, Avatar, SourceTag, Radar, Timeline, Stars, Score, Bar
<code>feedback_format.js</code>	SOURCE colours, <code>sourceColour()</code> , <code>formatDate()</code> , <code>relative()</code>
<code>feedback.css</code>	Every rule scoped under <code>.fb</code>

Why a slope graph for the reconciliation page

The question is "do these two instruments put the same people in the same order". A slope graph answers exactly that at a glance: a line runs from where the record places somebody to where the reviewers place them, so **flat lines are agreement and steep lines are disagreement**, and the direction of the slope says which instrument is the generous one. A scatter plot would answer the same question less directly and a table would not answer it at all.

Part C

Syntax and methods used, with what each one does

Everything listed here appears in the Budget or Feedback source. The counts are how many times each is used across the two features, so you can see which ones actually matter.

C1. JavaScript language features

Syntax	What it does	Example from this code
=> (arrow function)	A shorter way to write a function. Also keeps the surrounding this	<code>const money = (n) => Math.round(n * 100) / 100</code>
async / await	Marks a function as asynchronous and waits for a promise to settle, so database calls read top to bottom instead of nesting callbacks	<code>const budget = await ProjectBudget.findOne({...})</code>
?. (optional chaining)	Reads a property only if the thing before it exists. Returns undefined instead of throwing	<code>row.budget?.currency</code>
?? (nullish coalescing)	Uses the right-hand value only when the left is null or undefined. Unlike , a valid 0 is kept	<code>entry.costRateOverride ?? budget.projectCostRate ?? 0</code>
... (spread / rest)	Copies the contents of an array or object into a new one, or collects extra arguments	<code>{ ...person, recordPercentile }</code>
Destructuring	Pulls named values out of an object or array in one statement	<code>const { objective, budget, forecast } = data</code>
Template literal	A string in backticks that can embed expressions	<code>`\${hours}h at \${rate} an hour`</code>
Ternary ? :	A one-line if / else that produces a value	<code>willOverrun ? 'over' : 'ok'</code>

C2. Array methods

Method	What it does	Used here for
<code>.map()</code>	Makes a new array by transforming every item	Turning raw time entries into priced entries (201 uses)
<code>.filter()</code>	Makes a new array of only the items that pass a test	Keeping only the last 14 days of entries (112 uses)
<code>.reduce()</code>	Boils an array down to a single value	Summing cost, hours and billed amounts (45 uses)
<code>.sort()</code>	Reorders an array in place using a comparison	Worst overrun first, biggest gap first (47 uses)
<code>.find()</code>	Returns the first item that matches, or undefined	The rate in force on a given date (43 uses)
<code>.some()</code>	True if at least one item matches	Whether any action is an overload finding
<code>.slice()</code>	Copies part of an array without changing the original	The last 40 ledger rows for the table (30 uses)
<code>.join()</code>	Turns an array into a string with a separator	Building the email text (28 uses)
<code>.includes()</code>	True if the array contains a value	Whether a threshold has already fired
<code>.push()</code>	Adds to the end of an array	Collecting findings as they are discovered (53 uses)
<code>.reverse()</code>	Flips the order	Showing the newest ledger entries first

C3. Object, Map and Set

Method	What it does	Used here for
<code>new Map()</code>	A key-value store that keeps insertion order and allows any key type	Grouping entries by person or by day (24 uses)
<code>new Set()</code>	A collection with no duplicates	Unique days worked, unique threshold values (15 uses)
<code>.get()</code> / <code>.set()</code>	Read from and write to a Map	Running totals per person while looping once
<code>.has()</code>	Whether a Map or Set contains a key	Whether a person has already worked on a project
<code>Object.entries()</code>	Turns an object into an array of [key, value] pairs so it can be looped	Walking a proposed department plan
<code>Object.fromEntries()</code>	The reverse — builds an object from pairs	Rebuilding the blended-rate lookup
<code>Object.keys()</code> / <code>.values()</code>	The keys or the values of an object as an array	Iterating competency scores

C4. Math and numbers

Method	What it does	Used here for
<code>Math.round()</code>	Nearest whole number	Rounding money to cents and percentages to whole numbers (42 uses)
<code>Math.max()</code> / <code>Math.min()</code>	Largest or smallest of the arguments	Clamping the confidence spread between 8% and 42%
<code>Math.abs()</code>	Removes the sign	Ranking gaps by size regardless of direction
<code>Math.sqrt()</code>	Square root	Standard deviation, for the confidence range and for Spearman
<code>Math.floor()</code> / <code>Math.ceil()</code>	Rounds down or up	Percentile index; days to finish always rounds up
<code>.toFixed(n)</code>	Formats a number to n decimal places, as a string	Compact money like \$12.5k
<code>.toLocaleString()</code>	Formats a number with thousands separators	\$13,446 rather than 13446

C5. Mongoose — talking to MongoDB

Method	What it does	Used here for
<code>Model.find(filter)</code>	Every document matching a filter	All time entries for a project (43 uses)
<code>Model.findById(id)</code>	One document by its <code>_id</code>	Loading one project or one review (16 uses)
<code>Model.findOne(filter)</code>	The first document matching a filter	The budget for a project; the open shift for a person
<code>Model.create(doc)</code>	Inserts a new document	A new time entry, rate or review
<code>doc.save()</code>	Writes changes to an already-loaded document back to the database	Closing a shift; recording fired thresholds
<code>.populate(path, fields)</code>	Replaces a stored reference id with the actual document	Turning an employee id into a name and colour (20 uses)
<code>.select(fields)</code>	Asks for only certain fields, so less data crosses the wire	Loading just name and email for the alert (15 uses)
<code>.sort({field: 1})</code>	Orders the results in the database rather than in JavaScript	Rates newest-first, entries oldest-first
<code>Model.updateOne(filter, update)</code>	Changes matching documents without loading them	Resetting fired thresholds
<code>Model.countDocuments()</code>	Counts without fetching	How many employees have an email address

Method	What it does	Used here for
<code>Model.distinct(field)</code>	The unique values of one field	Which tasks have any time entries at all
<code>Model.aggregate([...])</code>	A pipeline that groups and totals inside the database	Summing hours per task in one query rather than in a loop
<code>Model.bulkWrite(ops)</code>	Many writes in a single round trip	Repairing spentHours across many tasks at once

Promise.all — worth knowing why it is there

`await Promise.all([a(), b(), c()])` starts several database queries at once and waits for all of them, instead of waiting for each in turn. The portfolio page originally loaded every project one after another, and each one re-fetched the whole rate table and employee list — twenty-four round trips to Atlas to draw one screen. Loading the shared data once and running the projects in parallel took it from 671ms to 401ms.

C6. Express — the HTTP layer

Method	What it does
<code>express.Router()</code>	A mini application that groups related routes, mounted under one path prefix
<code>router.get / post / patch / delete</code>	Registers a handler for one HTTP method and path
<code>app.use(path, router)</code>	Mounts a router, so <code>/api/budget + /portfolio</code> becomes the full path
<code>req.params</code>	Values from the URL, such as the <code>:id</code> in <code>/project/:id</code>
<code>req.query</code>	Values from the query string, such as <code>?cycle=2026-Q3</code>
<code>req.body</code>	The JSON sent with a POST or PATCH
<code>res.json(data)</code>	Sends a JSON response
<code>res.status(code)</code>	Sets the HTTP status before sending
<code>next(err)</code>	Passes an error to the shared error handler instead of crashing the request

Both controllers wrap every handler in a small `asyncRoute()` helper. An async function that throws returns a rejected promise, and Express 5 will not catch that on its own, so the helper attaches a `.catch(next)` to every route. Without it a database error would hang the request instead of returning a clean 500.

C7. React — the frontend

Hook or API	What it does	Used here for
<code>useState(initial)</code>	Holds a value that, when changed, re-renders the component	The loaded data, which drawer is open, form fields (98 uses)
<code>useEffect(fn, deps)</code>	Runs a side effect after render, re-running when the dependencies change	Fetching from the API when the page mounts (29 uses)
<code>useCallback(fn, deps)</code>	Keeps the same function identity between renders	The <code>load()</code> function, so the effect does not loop (28 uses)
<code>useMemo(fn, deps)</code>	Caches an expensive calculation until its inputs change	The live language check on the review form (10 uses)
<code>createContext / useContext</code>	Shares a value down the tree without passing props through every level	Who is "acting as", available to every page in the feature
<code>useParams()</code>	Reads the <code>:id</code> from the current route	Which project or which person to load
<code>useNavigate()</code>	Moves to another route from code rather than from a link	Opening a project when the whole card is clicked
<code><Link to="..."></code>	A navigation link that does not reload the page	Every cross-page link in both features

Hook or API	What it does	Used here for
Conditional rendering	{open && <div/>} renders only when the condition holds	Drawers, empty states, error banners
key prop	Gives React a stable identity for each item in a list so it can update efficiently	Every .map() that renders rows or cards

One React rule worth quoting

A value that can be worked out from existing state should be **derived during render**, not stored in state and synchronised with an effect. The review form originally used two effects to write back into its own form state, which is a render loop wearing a disguise: it set state, re-rendered, and the effect ran again to check its own work. Both are now single derived constants.

Part D

Questions you are likely to be asked

On the Budget feature

Why is the forecast built from 14 days rather than the whole project?

Because a lifetime average is dragged towards whatever the team was doing months ago. A project can look completely calm on its lifetime average while the last fortnight has already decided the outcome. Fourteen days is long enough to survive one quiet Friday and short enough that a change of pace shows up while it still matters.

Why show a range instead of a single predicted number?

Because a single number is a confidence nobody has earned. The range comes from how variable the daily burn has actually been — the standard deviation divided by the mean. A steady project earns a narrow band and an erratic one earns a wide one. Saying \$11,000 to \$12,100 is less impressive than saying \$11,568 and far more honest, because it tells the manager whether the number is worth acting on.

Why do you store rates in a separate collection instead of a field on the employee?

Because a single field on the employee would be edited when somebody is promoted, and that would silently rewrite what last quarter's hours cost — changing margin on projects that already closed. A dated Rate row means a rise applies only from the day it happened. I can demonstrate it: work logged 22 days ago still costs \$42 an hour while work logged 21 days ago costs \$50.

Why is the hard stop off by default?

Because refusing to record work somebody genuinely did makes the ledger wrong, and a wrong ledger is usually worse than an overrun — every forecast built on it afterwards is also wrong. It exists for fixed-fee contracts where the cap is contractual, and it is opt-in.

How do you make sure an alert fires only once?

The budget document keeps two lists: the thresholds being watched, and the ones already fired. After time is logged the current percentage is compared against both, and only newly crossed levels are recorded and announced. If several are crossed at once only one email goes out, for the highest, but all of them are marked fired so none can fire again.

What does the AI actually do here?

It phrases one summary paragraph and nothing else. It is sent only the findings the code has already computed — never the raw ledger — so it has nothing to invent a figure from. Without an API key a rules-based writer produces the same summary and the page says which one wrote it. Earlier the page reported a model name whenever a key existed while never actually calling it, which was a claim the code had not earned, so that was fixed.

On the Feedback feature

Why are all four review sources in one collection?

Because the comparison between them is the finding. If manager, peer, self and client feedback live in four tables or four tabs, you can only read them one at a time. In one collection with a source field they can be drawn on one set of axes, and then the gap between what somebody says about themselves and what everybody else says becomes visible — which is the single most useful thing the data contains.

How does calibration decide a reviewer is harsh?

Not against the company mean — that was the first version and it was wrong, because it flags anybody who happens to review a weak team. It compares a reviewer against what other reviewers gave the same people. Their own earlier reviews are excluded from that baseline, otherwise they are partly compared against themselves and every drift figure is dragged towards zero.

What stops the agent from writing rubbish into the system?

It cannot write at all. It produces a proposal with the evidence attached, and a named human approves or dismisses it. Only on approval is an objective created. Every step goes into an append-only audit log with who acted and when, and a fingerprint stops the same theme being raised twice. It also proposes only the strongest theme per person, because a development plan with six items is not a plan.

Why convert to percentiles instead of just scaling the rating to 100?

Because ratings cluster. Almost nobody is given a 1 or a 5, so an average of 3.6 is not 72% of anything — it is the middle of the range people actually use. Multiplying by 20 and putting it beside a score out of 100 would manufacture disagreement out of nothing but the shape of the two distributions. Percentiles within the company are unit-free and survive a lenient or harsh review culture.

Why Spearman rather than Pearson?

Because I am not claiming the two scales are linearly related — only that two instruments measuring anything alike should put people in a similar order. Spearman correlates the ranks, which is exactly that claim and no more. Pearson would assume a straight-line relationship between a derived score and a human rating, which there is no reason to expect.

What if the two disagree about somebody and nothing explains it?

The page says so. It reports that none of the usual causes hold — they are not overloaded, not slipping, and the ratings are not concentrated in one competency — and that this makes it a question for a conversation rather than for another query. Padding the card with a plausible sentence would be worse than useless.

On both**Why can no existing product do the cross-feature comparisons?**

Because the industry is split down exactly that line. Review platforms hold the human half and have no record of what anybody shipped. Work trackers hold the recorded half and have never heard of a review cycle. Time and billing tools own cost but not deadlines. This system holds the tasks, the time, the rates and the reviews in one database, so it can ask questions that need two halves at once — what a deadline change costs, or whether the reviewers and the delivery record agree about the same person.

How is the styling kept from breaking the rest of the app?

Every rule in each feature's stylesheet is scoped under a wrapper class — `.bud` for budget and `.fb` for feedback. The application already carries Bootstrap, the team's global stylesheet and two other feature stylesheets, and all of them define `.card` and `.btn`. Without the scope, whichever file the bundler emitted last would decide what the leave form looks like. The stylesheet is also imported inside the layout component, so it does not load at all until one of those pages is opened.

Give an example of a performance problem you found and fixed.

Listing rates fetched every rate row with its employee populated, and then re-fetched each row one at a time to format it — fifteen round trips to Atlas for data already in memory. Making the formatter a pure function over the document already in hand reduced it to one query. The same class of mistake appeared in the advisor, which re-queried the tasks that had already been loaded a moment earlier.

The sentence to have ready

"Both features were built to be defended rather than demonstrated. Every number on screen can be traced back to the records that produced it, the AI is only ever allowed to phrase and never to calculate, and where the data cannot answer a question the interface says so instead of guessing."